DEVELOPER TOOLS

AI · ML · DS / REVISION NOTES

AI-ML-DS

# Repository Guidelines

Structure, environments, revision PDFs and contribution conventions

4
PAGES

Compiled October 2026
github.com/ankitgupta1729/AI-ML-DS

# Contents

Project Overview 3

Project Structure & Module Organization 3

Build, Test, and Development Commands 3

- Main Python Environment 3
- Sub-module Environments 4
- Revision PDFs 4

Coding Style & Naming Conventions 4

Commit & Pull Request Guidelines 4

## Project Overview

This repository is a comprehensive collection of course notes, tutorials, practice exercises, and utilities covering AI, Machine Learning, Data Science, and modern software engineering tools. It includes hands-on materials for Python, PySpark, SQL, Go, Docker, TensorFlow, PyTorch, LangChain, LangGraph and LlamaIndex, plus a designed revision PDF for every course.

## Project Structure & Module Organization

Material is grouped into domain folders; each holds numbered, self-contained modules ( `<category>/<NN>.<Module-Name>/` ):

- `foundations/` : `01.Python` , `03.SQL` , `04.Theory` , `13.Decorators` , `45.Python-for-Everybody-…`
- `data-engineering/` : `02.PySpark` , `14.Databricks`
- `machine-learning/` : `33.AI-Nano-Degree` , `47` – `49` (Andrew Ng ML Specialization), `50.Andrew-Ng-Course-Material` (third-party repos)
- `deep-learning/` : `51` – `59` (TensorFlow, PyTorch, CNNs, GANs, computer vision)
- `nlp/` : `60` , `61`
- `generative-ai/` : `05.Code` , `17.OpenCode` , `20.LlamaIndex` , `22` – `30` , `32` , `40` , `46` (LLMs, RAG, LangChain, LangGraph, agents, `25.Claude-Code-Deep-Dive` )
- `mlops/` : `31` , `34` – `36` , `39` , `43` , `44` (ML in production, MLflow, CI/CD, GCP)
- `dev-tools/` : `09` – `12` , `15.uv_demo` , `21.pipenv-course` , `37` , `38`
- `programming-languages/` : `19.Go` , `41` , `42`
- `career/` : `06` – `08` , `16`
- `projects/18.Utilities` : helper scripts such as `images_to_pdf.py` and `send-mail.py` , chatbots, a website
- `resources/` : reference books and links; `docs/` : repository-level guides
- `tools/` : repository tooling — `notes-to-pdf/` (revision-PDF builder), `spelling/` (reviewed spelling and technical-term corrections for the notes) and `migrate-local-layout.sh`

Inside a course module: `Note.md` (source notes), `images/` (screenshots), `pdf/` (generated revision PDFs), plus notebooks and code.

## Build, Test, and Development Commands

The repository uses multiple dependency management systems depending on the module.

### Main Python Environment

- **Install dependencies**: `uv sync` , `pipenv install` or `pip install -r requirements.txt`
- **Python version**: 3.12 ( `.python-version` , `pyproject.toml` ); the legacy `Pipfile` targets 3.11

### Sub-module Environments

- **uv (`dev-tools/15.uv_demo`)**: Uses `uv` for fast environment management.
- **Pipenv**: Several modules (e.g. `generative-ai/20.LlamaIndex`, `generative-ai/23.LangChain`) have their own `Pipfile`.
- **Go (`programming-languages/19.Go`)**: Standard Go commands like `go run <file>.go`.

### Revision PDFs

- **Rebuild**: `python tools/notes-to-pdf/build.py [module-number …]` (needs Tesseract, Node + Playwright; see `tools/notes-to-pdf/README.md`).
- **Verify**: `python tools/notes-to-pdf/build.py --verify` checks every generated PDF.
- **Register a new course**: add an entry to `tools/notes-to-pdf/catalog.json`.
- **Docs checks**: `python tools/check_links.py` and `ruff check --config tools/ruff.toml tools/` (both run in CI).
- **Spelling**: fixes to course notes go into `tools/spelling/corrections.tsv` (reviewed, scoped per module) and are applied with `python tools/spelling/apply.py`.
- Never edit files in `pdf/` by hand, and never rewrite course `.md` notes to suit the PDF builder — the builder reads them as they are.

## Coding Style & Naming Conventions

- **Language Dominance**: Python is the primary language, often used in Jupyter Notebooks (`.ipynb`) for interactive learning.
- **Editor settings**: `.editorconfig` defines whitespace rules; recommended VS Code settings live in `.vscode/settings.json`, with `python.terminal.activateEnvironment` disabled to allow manual control.
- **New modules**: take the next free number and go into the matching category folder.

## Commit & Pull Request Guidelines

- **Commit Messages**: Use concise, descriptive messages. Common patterns in the repo include "added content" or "Add [topic] setup".
- **Workflow**: Direct commits are common for adding new learning materials. No specific PR template is enforced; see `CONTRIBUTING.md`.